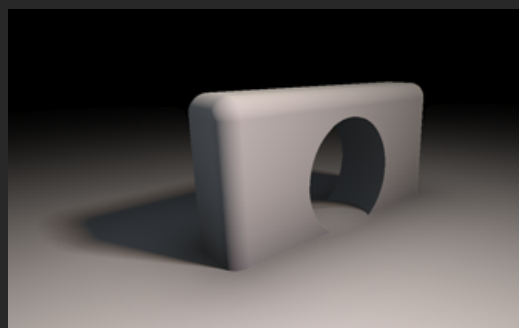


Inigo Quilez :: articles :: soft shadows in raymarched SDFs - 2010

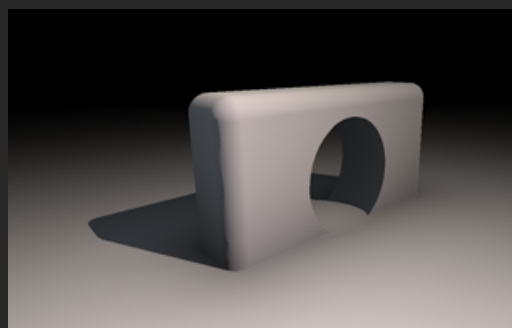


intro

One of the many advantages of **SDFs** (Signed Distance Fields), is that querying global information is easy. What I mean is that, when shading a point, one can easily explore the surrounding geometry by simply querying the distance function at the desired locations. Compare that to a classic rasterizer (REYES or tile based), where polygons are processed independently without knowledge of each other, and the only way to access other polygons than the one currently being processed is baking them somehow as a preprocess (in a shadowmap, a depthmap, a pointcloud...). In a classic raytracer, finding global information is a bit easier, since the data is usually ready in memory or at least one geometry cache-miss away. However in this case exploring the geometry still requires expensive computations like casting multiple rays. However, with SDFs (at least the procedural type we are talking about here), the distance field itself is available and ready for querying at any time, at any location, without any extra passes or effort. This means that we should think of new ways to exploit global information about the scene and use it for, say, more realistic shading and illumination techniques. In this article we are going to do exactly that and render soft shadows with penumbra, virtually for free, when raymarching SDFs.



Soft shadow and penumbra computed for free



Classic shadow raycasting

First Attempt

So, let's assume you have a distance field encoded in function `float map(vec3 p)` (you can learn a bit about how to construct some basic distance functions [here](#)). For simplicity, let's assume that this `map()` function contains all of the world description in it, and that all objects are allowed to cast shadows on all other objects. With these assumptions, the easy way to compute shadowing information at a point is to raymarch along the light vector, as far as the distance from the light to the shading point is, until an intersection is found, if any. You can probably do that with some code like this:

```
float shadow( in vec3 ro, in vec3 rd, float mint, float maxt )
{
    float t = mint;
    for( int i=0; i<256 && t<maxt; i++ )
    {
        float h = map(ro + rd*t);
        if( h<0.001 )
            return 0.0;
        t += h;
    }
    return 1.0;
}
```

This code works beautifully, and produces nice and accurate shadows, as seen in the rightmost image at the beginning of this article. Now, thanks to SDFs, with a single additional line of code we can make this look much better. To make that happen, we need to think about what happens when a shadow ray doesn't hit any object yet is pretty close to doing so. Because in that case, we might want to shade our point with a partial shadow, a penumbra. And in terms of choosing how dark or bright that partial shadow should be, it makes sense that the closer the ray is to hitting an object, the darker the penumbra should be. At the same time, it also makes sense that the closer this near-hit happened from the point you are shading, the darker too. We can then propose the following shadow formula:

shadow &proportional; closest_miss / distance_to_closest_miss

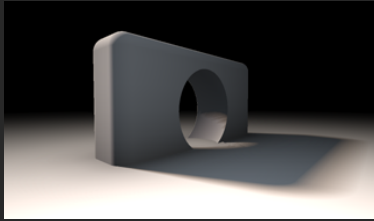
With the symbol &proportional; meaning “proportional to”. Well, it happens that as we raymarch our shadow ray, both these quantities are available to us! Of course the first one is `h` in the code above, and the second one is `t`. So, we can simply compute a penumbra factor for every step point in our marching process and take the darkest of all penumbras.

```
float softshadow( in vec3 ro, in vec3 rd, float mint, float maxt, float k )
{
    float res = 1.0;
    float t = mint;
    for( int i=0; i<256 && t<maxt; i++ )
    {
```

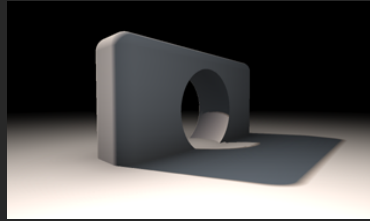
```

float h = map(ro + rd*t);
if( h<0.001 )
    return 0.0;
res = min( res, k*h/t );
t += h;
}
return res;
}

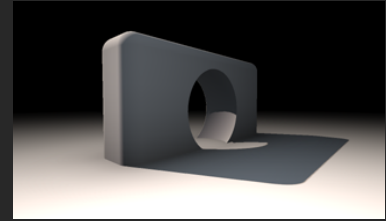
```



k = 8



k = 32



k = 128

This simple modification is enough to generate the much nicer left image in the beginning of this page. As you can see, the improvement is massive: not only do you get soft shadows, but they even behave realistically in that shadows are sharper when close to the occluder contact (see where the bridge touches the floor) and the penumbras soften away from the occluder. This is a rather desirable effect that usually requires casting dozens or hundreds of rays per pixel, while here we are doing with a single ray, and virtually for free (given the cost of the extra computation we just introduced is negligible compared to the cost of evaluating **map()**).

The parameter **k** controls how hard/soft the shadows are, as you can see in the images above comparing the same shadow rendered with different values of **k**. We'll come back to this parameter soon, but for now, here is an example of the technique in action, in a raymarched procedural distance field:



Soft penumbra shadows in action, year 2010

You can see a lot more examples of this technique in action in the [raymarching with signed distance fields](#) article.

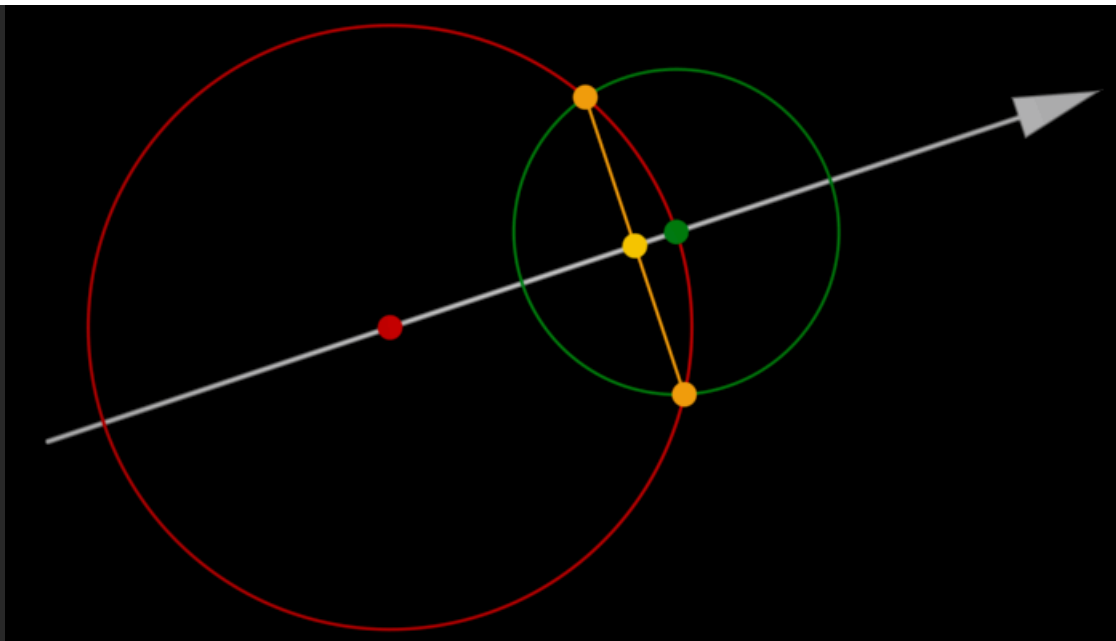
Light size

So, on the parameter k , as we know, the larger it is, the sharper the shadow becomes. So it doesn't take much of a leap to hypothesize whether it is related to the inverse of the light source's size (as a solid angle), since the larger the light size, the softer its shadows. And indeed, as can be seen in Figure 2 below, that is the case (top left is the SDF soft shadow presented here, the top right is the groundtruth; bottom row is the same comparison for a larger light source).

An improvement

7 years after the publication of this technique, Sebastian Aaltonen published an improvement at his GDC presentation, that helps some of the banding artifacts you can get from this technique, especially for shadows from caster with sharp corners.

Beware that for this algorithm to be stable, we should be searching for penumbras exhaustively along the ray. However since we are marching, chances are we miss the point along the ray that produces the darkest penumbra. That can manifest as light leaking in patterns that match the marching steps. In particular, sharp corners in the shadow casters are a usual source of missed penumbras. Sebastian's technique helps with the situation by computing the penumbra as h/t not just at the ray marching sampling positions, but at an estimation of the closest point from the surface to the marched ray at each iteration. Or in other words, by using the current sampling point and the previous one, his technique computes a closest distance estimation by triangulating the information. The picture below shows the geometry of the situation:



The white arrow is the ray we are marching. The green dot is the current position along the ray, and the red dot is the previous position. The green and red circles represent the current and previous SDF unbounding spheres. One can estimate that the closest surface will be at a point close to where these two spheres meet (yellow line and pair of dots). The closest point along the ray will be the intersection of that yellow area with the actual ray (yellow dot in the center).

Let's call y the distance from the current point (green) to that closest point along the ray (yellow), and d the distance from that point to the estimated closest distance (half the length of the yellow line in the diagram above). Then, the code to compute these two quantities is pretty easy:

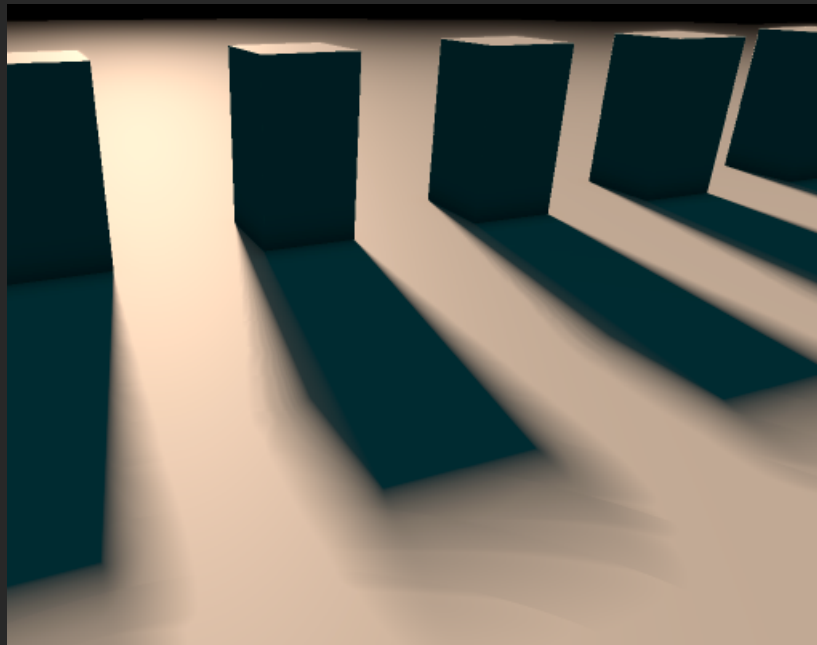
```
float y = r2*r2/(2.0*r1);
float d = sqrt(r2*r2-y*y);
```

where $r1$ and $r2$ are the radius of the red and green sphere, or in other words, the SDFs evaluation at the previous and current raymarch points. From these two quantities, we can improve our penumbra shadow estimation by doing:

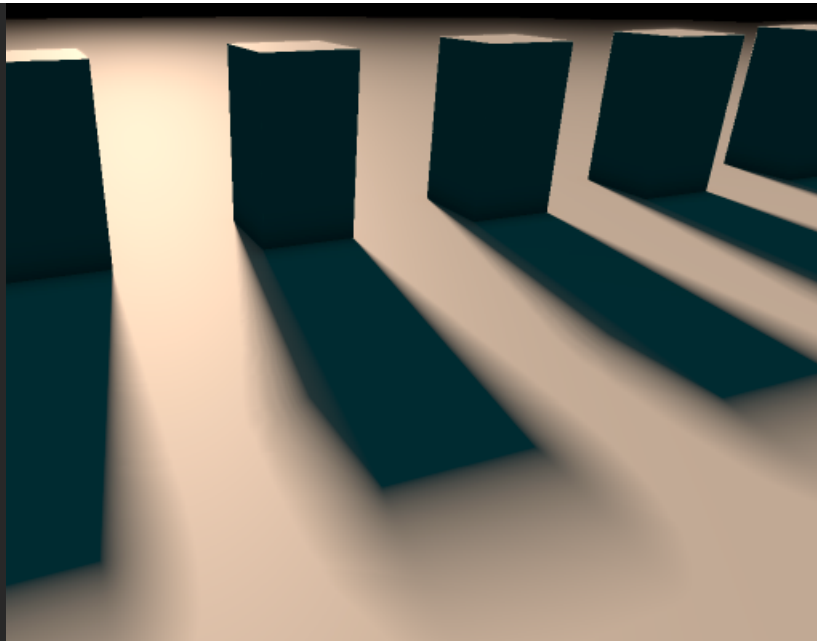
```
float softshadow( in vec3 ro, in vec3 rd, float mint, float maxt, float w )
{
    float res = 1.0;
    float ph = 1e20;
    float t = mint;
    for( int i=0; i<256 && t<maxt; i++ )
    {
        float h = map(ro + rd*t);
```

```
    if( h<0.001 )  
        return 0.0;  
    float y = h*h/(2.0*ph);  
    float d = sqrt(h*h-y*y);  
    res = min( res, d/(w*max(0.0,t-y)) );  
    ph = h;  
    t += h;  
}  
return res;  
}
```

This produces better shadows in difficult cases, as can be seen in the comparison below:



Original method



Improved method

You can find reference implementation for the improved method here: <https://www.shadertoy.com/view/lsKcDD>

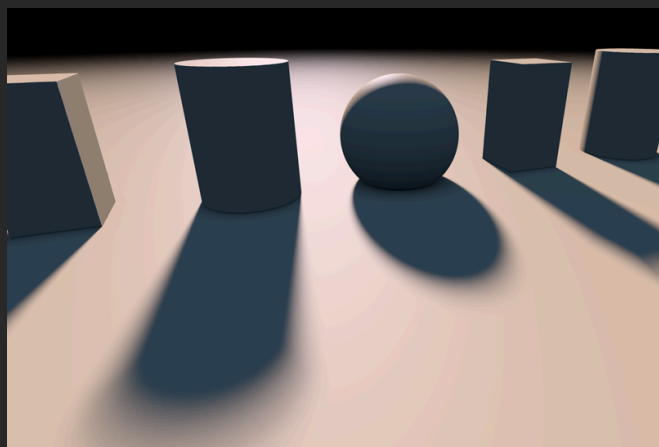
Another improvement

Some time later, Shadertoy user “nurof3n” explored ways to extend the original technique to capture the inner penumbras of shadows by letting the ray go inside the geometry in order to capture near-misses, on top of the near-hits. I expanded on his idea and came up with this simple extension to the original technique, that captures the inner penumbras:

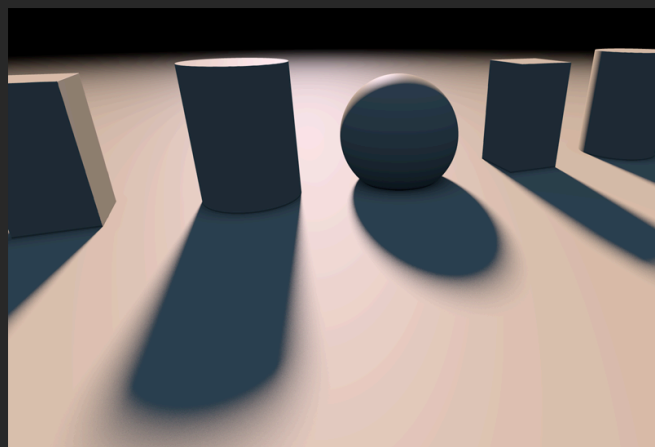
```
float softshadow( in vec3 ro, in vec3 rd, float mint, float maxt, float w )
{
    float res = 1.0;
    float t = mint;
    for( int i=0; i<256 && t<maxt; i++ )
    {
        float h = map(ro + t*rd);
        res = min( res, h/(w*t) );
        t += clamp(h, 0.005, 0.50);
        if( res<-1.0 || t>maxt ) break;
    }
    res = max(res, -1.0);
    return 0.25*(1.0+res)*(1.0+res)*(2.0-res);
}
```

Three notes. First, the w is, again, the light size (solid angle). Second, we no longer break the loop on negative distances (h), but let the marchers continue through the inside of the SDF until the shadow is really dark (-1). For that to work and prevent back-marching, we need to take increments of h that are always positive, so either use $\text{abs}(h)$ as marching step or do some clamping as I show in the code above (exact values to be determined based on the scale of your scene). And third, the last line implements a smoothstep of the range -1 to 1 , rather than the usual 0 to 1 , and therefore takes this unusual form.

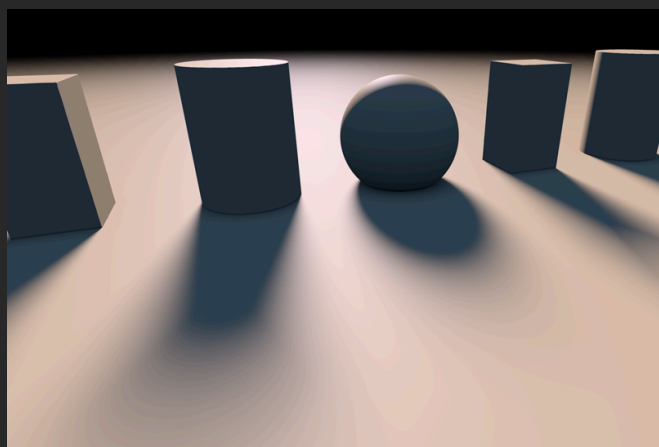
With this adjustment, the final SDF shadows look pretty close to the physically correct ground truth in terms of behaving in plausible way:



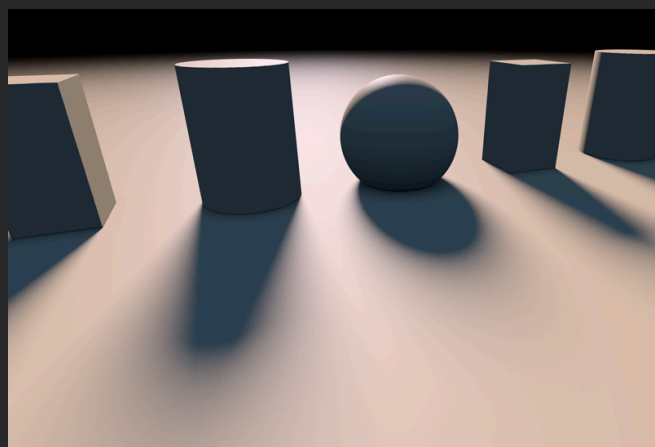
SDF shadows, small light source (w)



Reference, small light source (w)



SDF shadows, large light source (w)



Reference, large light source (w)